



**UNIVERSIDAD
DON BOSCO**

Semana 9

ADO.NET — Acceso a Datos Conectado



SqlConnection · SqlCommand · SqlDataReader

Bases de Datos y SQL

SqlConnection

SqlCommand

SqlDataReader

Contenido y ritmo de la clase

01	Bases de datos y SQL	Tablas, sentencias, SELECT, WHERE, INSERT	28 min
02	ADO.NET y la forma conectada	El modelo de acceso a datos en .NET	12 min
03	SqlConnection	Connection string y el patrón using	12 min
04	SqlCommand y parámetros	Ejecutar SQL de forma segura	18 min
05	SqlDataReader	Leer resultados fila por fila	12 min
06	Ejercicio guiado paso a paso	Lo construimos juntos, línea por línea	25 min
07	Ejercicio práctico	Ustedes lo hacen; yo acompaño	25 min

+ 10 min de receso entre el bloque 05 y el ejercicio guiado

¿Qué van a poder hacer al terminar?

1

Escribir una sentencia SQL

Construir SELECT, INSERT, UPDATE y DELETE identificando cada cláusula y su orden obligatorio.

2

Conectar C# con la base de datos

Armar un connection string y abrir/cerrar la conexión con el patrón using.

3

Ejecutar SQL de forma segura

Usar SqlCommand con parámetros y elegir entre ExecuteReader, ExecuteNonQuery y ExecuteScalar.

4

Leer los resultados

Recorrer un SqlDataReader con while (lector.Read()) y extraer cada columna con el tipo correcto.



Al final deben poder abrir Visual Studio, escribir 20 líneas y ver datos reales impresos en pantalla.

01

Bases de Datos y SQL

El lenguaje para hablar con los datos

28 min

¿Qué es una base de datos relacional?



Una base de datos es un conjunto de TABLAS organizadas, como hojas de cálculo relacionadas entre sí. Cada tabla tiene columnas (campos) y filas (registros), y el motor las guarda de forma permanente en disco.

Tabla: Productos

Id	Nombre	Precio	Stock
1	Laptop	599.99	10
2	Mouse	12.99	50
3	Teclado	29.99	25

↑ *columna (campo)*

← *fila (registro)*

Vocabulario clave

Tabla

Colección de datos del mismo tipo (Productos, Clientes)

Columna / Campo

Un atributo del elemento (Nombre, Precio)

Fila / Registro

Un elemento individual completo

Clave primaria (PK)

Columna que identifica de forma única cada fila (Id)

Anatomía de una sentencia SQL



Una SENTENCIA SQL es una instrucción completa que se le envía al motor de base de datos. Se arma juntando CLÁUSULAS en un orden fijo, y termina con punto y coma. No es código C#: es un texto que viaja hasta la base de datos y allá se ejecuta.

```
SELECT Nombre, Precio FROM Productos WHERE Stock > 0 ORDER BY Precio DESC ;
```

SELECT

cláusula

Qué columnas quiero traer

FROM

cláusula

De qué tabla las traigo

WHERE

cláusula

Qué filas cumplen la condición

ORDER BY

cláusula

En qué orden las devuelvo

;

terminador

Marca el fin de la sentencia



El orden de las cláusulas NO es opcional: SELECT → FROM → WHERE → ORDER BY. Si las escribes en otro orden, el motor rechaza la sentencia con un error de sintaxis.

Construir un SELECT paso a paso

Empezamos con lo mínimo y le vamos agregando cláusulas:

1

Traer todo

El asterisco significa 'todas las columnas'. Devuelve la tabla completa.

```
SELECT * FROM Productos;
```

2

Elegir columnas

Se listan separadas por coma. Trae menos datos y es más rápido.

```
SELECT Nombre, Precio FROM Productos;
```

3

Filtrar filas

WHERE deja pasar solo las filas que cumplen la condición.

```
SELECT Nombre, Precio FROM Productos WHERE Stock > 0;
```

4

Ordenar

ASC = de menor a mayor (por defecto). DESC = de mayor a menor.

```
SELECT Nombre, Precio FROM Productos WHERE Stock > 0 ORDER BY  
Precio DESC;
```



Regla práctica: piensa primero FROM (de dónde), luego SELECT (qué) y luego WHERE (cuáles).

La cláusula WHERE — filtrar filas



WHERE evalúa una CONDICIÓN fila por fila: si es verdadera la fila entra al resultado, si es falsa se descarta. Ojo: en SQL el texto va entre comillas SIMPLES — WHERE Nombre = 'Laptop'. Las comillas dobles son de C#, no de SQL.

Operadores de comparación

= Igual a

WHERE Precio = 20

<> ó != Distinto de

WHERE Stock <> 0

> < >= <= Mayor / menor

WHERE Precio >= 100

BETWEEN Entre dos valores

WHERE Precio BETWEEN 10 AND 50

LIKE Coincidencia de texto

WHERE Nombre LIKE 'Lap%'

Combinar y casos especiales

AND Se deben cumplir ambas

WHERE Stock > 0 AND Precio < 100

OR Basta con una

WHERE Stock = 0 OR Precio > 500

NOT Niega la condición

WHERE NOT Stock = 0

IN Está en una lista

WHERE Id IN (1, 3, 5)

IS NULL Campo sin valor

WHERE Precio IS NULL

Sentencias que modifican datos

INSERT — agregar una fila

```
INSERT INTO Productos (Nombre, Precio, Stock)
VALUES ('Monitor', 189.99, 15);
```

Las columnas y los valores deben ir en el MISMO orden y misma cantidad.

UPDATE — modificar filas existentes

```
UPDATE Productos
SET Precio = 179.99, Stock = 12
WHERE Id = 4;
```

SET indica qué cambiar. Sin WHERE, actualiza TODAS las filas de la tabla.

DELETE — eliminar filas

```
DELETE FROM Productos
WHERE Id = 4;
```

Sin WHERE, borra la tabla entera. No hay Ctrl+Z en una base de datos.



UPDATE y DELETE sin WHERE destruyen la tabla entera. Escribe SIEMPRE el WHERE primero.

02

ADO.NET

El puente entre C# y la base de datos

12 min

¿Qué es ADO.NET y la forma conectada?



ADO.NET es el conjunto de clases de .NET para hablar con bases de datos. La forma CONECTADA significa: abro la conexión, hago lo que necesito, y la cierro enseguida. Los datos no se quedan en memoria: cada vez que los necesito, vuelvo a preguntar.



1. Abrir conexión

```
SqlConnection.Open()
```



2. Ejecutar comando

```
SqlCommand.Execute...()
```



3. Leer resultados

```
SqlDataReader.Read()
```



4. Cerrar conexión

using → automático



Regla de oro: una conexión abierta que no se cierra consume recursos del servidor. Con muchos usuarios simultáneos eso tumba la aplicación. Por eso 'conectada' significa abrir-usar-cerrar RÁPIDO.



El patrón using de C# garantiza que la conexión se cierre SIEMPRE, incluso si ocurre un error a la mitad.

Las 3 clases de la forma conectada



SqlConnection

Representa la conexión física con la base de datos. Guarda el connection string: dónde está el servidor, cómo se llama la base y con qué credenciales entrar.

```
new SqlConnection(cadena)
```



SqlCommand

Representa una instrucción SQL lista para ejecutarse (SELECT, INSERT, UPDATE, DELETE), junto con sus parámetros y la conexión por la que viajará.

```
new SqlCommand(sql, conexion)
```



SqlDataReader

Lee el resultado de un SELECT, una fila a la vez y solo hacia adelante. Es la forma más rápida y ligera de recorrer muchos registros.

```
comando.ExecuteReader()
```



El flujo es siempre el mismo y es una cadena: Connection → Command → DataReader. Cada eslabón necesita al anterior; un Command sin conexión no puede ejecutarse, y un DataReader sin Command no existe.

03

SqlConnection

Abrir y cerrar la conexión correctamente

12 min

El connection string, parte por parte



El connection string es un texto con pares clave=valor separados por punto y coma. Le dice a .NET dónde está el servidor, qué base usar y cómo autenticarse.

```
string cadena = "Server=(localdb)\\mssqllocaldb;Database=TiendaDB;Trusted_Connection=True;";
```

Server

Dónde está el motor. (localdb)\mssqllocaldb es la instancia local que instala Visual Studio.

Database

El nombre de la base de datos dentro de ese servidor.

Trusted_Connection

True = entrar con el usuario de Windows, sin usuario ni contraseña.

User Id / Password

Solo si el servidor NO usa autenticación de Windows.



NUNCA escribas contraseñas en el código que subes a GitHub. Guárdalas en appsettings.json o en variables de entorno.

Abrir y cerrar con el patrón using

❌ Forma manual — propensa a errores

```
SqlConnection cn =  
    new SqlConnection(cadena);  
cn.Open();  
  
// ... trabajar con la base ...  
  
cn.Close();  
// si ocurre una excepcion antes  
// de esta línea, la conexion  
// queda ABIERTA para siempre
```

✅ Con using — forma correcta

```
using (SqlConnection cn =  
    new SqlConnection(cadena))  
{  
    cn.Open();  
  
    // ... trabajar con la base ...  
  
} // <- se cierra AQUI, siempre,  
// aunque haya habido un  
// error a la mitad
```

Versión moderna (C# 8 en adelante) — sin llaves:

```
using SqlConnection cn = new SqlConnection(cadena); // se cierra al terminar el metodo  
cn.Open();
```



using llama a Dispose(), que cierra la conexión: es un try/finally que no tienes que escribir.

04

SqlCommand

Ejecutar SQL de forma segura

18 min



Si pegas texto escrito por el usuario directamente dentro del SQL, el usuario puede escribir algo que cambie el SIGNIFICADO de tu consulta. Eso se llama inyección SQL.

❌ PELIGROSO — concatenar texto del usuario

```
string nombre = Request.Form["nombre"];          // el usuario escribe:  ' OR '1'='1
string sql = "SELECT * FROM Usuarios WHERE Nombre = '" + nombre + "'";
// SQL que llega al motor:
// SELECT * FROM Usuarios WHERE Nombre = '' OR '1'='1'
// '1'='1' es SIEMPRE verdadero -> devuelve TODOS los usuarios
```

✅ SEGURO — usar parámetros

```
string sql = "SELECT * FROM Usuarios WHERE Nombre = @nombre";
comando.Parameters.AddWithValue("@nombre", nombre);
// El motor trata @nombre SIEMPRE como un dato, nunca como código
```



Regla absoluta: nunca concatenes datos del usuario en un SQL con + ni con \$"". Siempre parámetros.

1

Escribir la sentencia SQL como texto, con @parámetros en los lugares donde irán datos variables.

```
string sql = "SELECT Nombre, Precio FROM Productos WHERE Precio > @min";
```

2

Crear el SqlCommand pasándole la sentencia y la conexión por la que viajará.

```
using SqlCommand cmd = new SqlCommand(sql, cn);
```

3

Darle valor a cada parámetro. El nombre debe coincidir exactamente con el del SQL.

```
cmd.Parameters.AddWithValue("@min", 20.0);
```

4

Ejecutar. El método que elijas depende de lo que devuelva la sentencia.

```
using SqlDataReader lector = cmd.ExecuteReader();
```



El parámetro lleva arroba en los dos lados: @min en el SQL y "@min" en AddWithValue. Si no coinciden, falla.

¿Cuál Execute uso? — los tres métodos

ExecuteReader ()

Cuando: SELECT que devuelve varias filas

Devuelve: Devuelve un SqlDataReader para recorrer los resultados.

```
string sql = "SELECT * FROM Productos";  
using SqlDataReader r =  
    cmd.ExecuteReader();
```

ExecuteNonQuery ()

Cuando: INSERT, UPDATE, DELETE

Devuelve: Devuelve un int: la cantidad de filas afectadas.

```
string sql = "DELETE FROM Productos WHERE Id=@id";  
int filas = cmd.ExecuteNonQuery();
```

ExecuteScalar ()

Cuando: Un solo valor: COUNT, SUM, MAX

Devuelve: Devuelve un object con la primera columna de la primera fila.

```
string sql = "SELECT COUNT(*) FROM Productos";  
int total = (int)cmd.ExecuteScalar();
```



Pregúntate qué te devuelve la sentencia: ¿muchas filas? Reader. ¿ningún dato, solo un cambio? NonQuery. ¿un número suelto? Scalar.

05

SqlDataReader

Leer los resultados fila por fila

12 min

¿Cómo funciona el SqlDataReader?



El DataReader es un cursor que avanza fila por fila, SOLO hacia adelante. Es rápido y usa poca memoria porque no carga todo de golpe: trae una fila, la lees, y pasa a la siguiente.

Resultado en la base de datos

Laptop — \$599.99

Mouse — \$12.99

← *aquí está el cursor*

Teclado — \$29.99

Monitor — \$189.99

El bucle que lo recorre

```
while (lector.Read())
{
    // cada Read() avanza UNA
    // fila y devuelve true

    string nombre =
        lector.GetString(0);

    // cuando ya no hay filas
    // devuelve false y sale
}
```



El DataReader exige que la conexión siga ABIERTA, y solo se recorre UNA vez hacia adelante.

Cómo sacar el valor de cada columna



Dentro del while, el lector representa la fila actual. Cada columna se pide por su POSICIÓN (empezando en 0) o por su NOMBRE.

```
string sql = "SELECT Id, Nombre, Precio FROM Productos"; // 0 1 2

while (lector.Read())
{
    int id = lector.GetInt32(0); // por posicion
    string nombre = lector["Nombre"].ToString(); // por nombre (mas legible)
    decimal precio = lector.GetDecimal(2);

    Console.WriteLine($"{id} - {nombre} - ${precio}");
}
```

`GetInt32 / GetString /
GetDecimal / GetDateTime`

Devuelven el valor ya convertido al tipo de C#. El tipo debe coincidir con el de la columna.

`lector["Columna"]`

Devuelve object: hay que convertirlo (ToString(), Convert.ToInt32). Más legible y no se rompe si cambia el orden.

`lector.IsDBNull(2)`

Comprueba si la columna viene vacía (NULL). Llamar a GetDecimal sobre un NULL lanza excepción.



Receso

10 minutos — al volver: abran Visual Studio

06

Ejercicio guiado

Lo construimos juntos, paso a paso

25 min

Qué vamos a construir



Una app de consola que se conecta a la base de datos y muestra los productos cuyo precio sea mayor a un valor dado. Unas 20 líneas.

Los 5 pasos

- 1 Crear la tabla y cargarle datos
- 2 Crear el proyecto e instalar el paquete
- 3 Escribir el connection string
- 4 Escribir el SQL y crear el comando
- 5 Ejecutar y leer con el DataReader

Resultado esperado en pantalla

```
--- PRODUCTOS (precio > 20) ---
```

1	Laptop	\$599.99	stock 10
3	Teclado	\$29.99	stock 25
4	Monitor	\$189.99	stock 15

```
3 producto(s) encontrado(s).
```



Si ven esos datos en la consola, su código C# viajó a la base de datos y trajo información real.

PASO 1

Crear la tabla y cargarle datos



En Visual Studio: Ver → Explorador de objetos de SQL Server → (localdb)\MSSQLLocalDB → clic derecho en Bases de datos → Agregar nueva base de datos → nómbrala TiendaDB. Luego clic derecho sobre ella → Nueva consulta y pega esto:

```
CREATE TABLE Productos (  
    Id          INT IDENTITY(1,1) PRIMARY KEY,    -- se numera solo: 1, 2, 3...  
    Nombre      NVARCHAR(100) NOT NULL,           -- texto obligatorio  
    Precio      DECIMAL(10,2) NOT NULL,           -- 10 digitos, 2 decimales  
    Stock       INT NOT NULL  
);  
  
INSERT INTO Productos (Nombre, Precio, Stock) VALUES  
    ('Laptop', 599.99, 10),  
    ('Mouse', 12.99, 50),  
    ('Teclado', 29.99, 25),  
    ('Monitor', 189.99, 15);  
  
SELECT * FROM Productos;    -- verificar que quedaron las 4 filas
```



Ejecuta con Ctrl+Shift+E. Si el SELECT final muestra 4 filas, la base está lista para el paso 2.

Crear el proyecto e instalar el paquete

1

Crear el proyecto

Archivo → Nuevo → Proyecto → Aplicación de consola (C#). Nómbralo CatalogoProductos.

2

Instalar el paquete de acceso a datos

Las clases SqlConnection, SqlCommand y SqlDataReader NO vienen incluidas: hay que agregar el paquete Microsoft.Data.SqlClient desde NuGet.

3

Importar el espacio de nombres

Al inicio del archivo Program.cs, para poder usar las clases por su nombre corto.

Lo que hay que escribir:

```
// En la Consola del Administrador de paquetes (Herramientas -> NuGet):  
Install-Package Microsoft.Data.SqlClient  
  
// Primera linea de Program.cs:  
using Microsoft.Data.SqlClient;
```



Si Visual Studio subraya SqlConnection en rojo, falta el paquete o falta esta línea using.

Paso 3 — la dirección de la base de datos

```
string cadena = "Server=(localdb)\\mssqllocaldb;Database=TiendaDB;Trusted_Connection=True;";
```

Paso 4 — la sentencia SQL y el comando que la ejecutará

```
// 1. La sentencia, con un parametro donde ira el dato variable
string sql = "SELECT Id, Nombre, Precio, Stock FROM Productos " +
             "WHERE Precio > @precioMinimo " +
             "ORDER BY Precio DESC";
// 2. Abrimos la conexion dentro de un using
using SqlConnection conexion = new SqlConnection(cadena);
conexion.Open();
// 3. Creamos el comando: sentencia + conexion
using SqlCommand comando = new SqlCommand(sql, conexion);
// 4. Le damos valor al parametro
comando.Parameters.AddWithValue("@precioMinimo", 20.0);
```



Ojo al orden: primero la conexión, después el comando. Y cuida el espacio al final de cada trozo del SQL.

Ejecutar y leer los resultados

```
// 5. Ejecutamos: como es un SELECT que devuelve filas, usamos ExecuteReader
using SqlDataReader lector = comando.ExecuteReader();

Console.WriteLine("--- PRODUCTOS (precio > 20) ---");
Console.WriteLine();

int contador = 0;

// 6. Read() avanza una fila y devuelve true mientras haya datos
while (lector.Read())
{
    int id          = lector.GetInt32(0);
    string nombre   = lector.GetString(1);
    decimal precio  = lector.GetDecimal(2);
    int stock       = lector.GetInt32(3);

    Console.WriteLine($"{id} {nombre,-12} ${precio,-9} stock {stock}");
    contador++;
}

Console.WriteLine($"{contador} producto(s) encontrado(s).");
```



Los números 0, 1, 2, 3 son las posiciones de las columnas EN EL SELECT, no en la tabla.

El programa completo

```
using Microsoft.Data.SqlClient;

// 1. Donde esta la base de datos
string cadena = "Server=(localdb)\\mssqllocaldb;" +
    "Database=TiendaDB;" +
    "Trusted_Connection=True;";

// 2. La sentencia SQL, con parametro
string sql = "SELECT Id, Nombre, Precio " +
    "FROM Productos " +
    "WHERE Precio > @precioMinimo " +
    "ORDER BY Precio DESC";

// 3. Conexion (using = se cierra sola)
using SqlConnection conexion =
    new SqlConnection(cadena);
conexion.Open();
```

```
// 4. Comando = sentencia + conexion
using SqlCommand comando =
    new SqlCommand(sql, conexion);

// 5. Valor del parametro
comando.Parameters
    .AddWithValue("@precioMinimo", 20.0);

// 6. Ejecutar: SELECT -> ExecuteReader
using SqlDataReader lector =
    comando.ExecuteReader();

// 7. Recorrer fila por fila
while (lector.Read())
{
    Console.WriteLine(
        $"{lector.GetInt32(0)} " +
        $"{lector.GetString(1)} " +
        $"{lector.GetDecimal(2)}");
}
```



Este es el esqueleto completo. Los 7 pasos se repiten SIEMPRE, sin importar la consulta: cadena → sentencia → conexión → comando → parámetro → ejecutar → recorrer.

07

Ejercicio práctico

Ahora les toca a ustedes

25 min

Enunciado — Consulta de estudiantes



Crea una app de consola que muestre los estudiantes de una carrera específica, ordenados por promedio. Tienen 25 minutos.

Preparen la tabla con este script:

```
CREATE TABLE Estudiantes (  
  Id INT IDENTITY(1,1) PRIMARY KEY,  
  Nombre NVARCHAR(100) NOT NULL,  
  Carrera NVARCHAR(50) NOT NULL,  
  Promedio DECIMAL(4,2) NOT NULL  
);  
INSERT INTO Estudiantes (Nombre, Carrera, Promedio) VALUES  
  ('Ana Lopez','Ingenieria',9.10), ('Luis Perez','Ingenieria',7.45),  
  ('Marta Diaz','Diseno',8.80),    ('Jose Cruz','Ingenieria',6.20),  
  ('Sofia Mena','Diseno',9.55);
```

Requisitos

Mostrar solo una carrera

Filtra con WHERE Carrera = @carrera usando un parámetro.

Ordenar por promedio

De mayor a menor.

Formato legible

Nombre, carrera y promedio en cada línea.

Contar al final

Cuántos estudiantes se mostraron.



Thank You
for your attention





UNIVERSIDAD DON BOSCO

udb.edu.sv



UDBelsalvador

